

5 Python Scripts Every Creator Needs

Practical local utilities for media and content workflows · DigitalToolsStudios

These are short, useful scripts you can copy into your own projects. They are not a product brochure. At the end there is a brief note about a related tool I maintain.

1. File Organizer by Extension

Moves files from a messy downloads folder into subfolders by extension (images, videos, documents, other).

```
import os, shutil
from pathlib import Path

SRC = Path.home() / "Downloads"
MAP = {
    ".jpg": "images", ".jpeg": "images", ".png": "images", ".webp": "images",
    ".mp4": "videos", ".mov": "videos", ".mkv": "videos",
    ".pdf": "docs", ".docx": "docs", ".txt": "docs",
}

for f in SRC.iterdir():
    if f.is_file():
        folder = MAP.get(f.suffix.lower(), "other")
        dest = SRC / folder
        dest.mkdir(exist_ok=True)
        shutil.move(str(f), dest / f.name)
        print(f"Moved {f.name} → {folder}/")
```

2. Batch Image Renamer

Renames all images in a folder to a clean prefix + sequential number. Useful before uploading or archiving.

```
from pathlib import Path

FOLDER = Path("./images")
PREFIX = "shot"
exts = {".jpg", ".jpeg", ".png", ".webp"}

files = sorted([f for f in FOLDER.iterdir() if f.suffix.lower() in exts])
for i, f in enumerate(files, 1):
    new_name = f"{PREFIX}_{i:03d}{f.suffix.lower()}"
    f.rename(FOLDER / new_name)
    print(new_name)
```

3. CSV Content Tracker

Keeps a simple CSV of content pieces with status, platform and date. No database required.

```
import csv
from datetime import date
from pathlib import Path

CSV = Path("content_tracker.csv")
HEADER = ["title", "platform", "status", "date"]

if not CSV.exists():
    with CSV.open("w", newline="", encoding="utf-8") as f:
        csv.writer(f).writerow(HEADER)

def add(title, platform, status="draft"):
```

```

with CSV.open("a", newline="", encoding="utf-8") as f:
    csv.writer(f).writerow([title, platform, status, date.today().isoformat()])
print("Added:", title)

# example
add("Weekly tip video", "TikTok", "ready")
add("Carousel about local tools", "Instagram", "draft")

```

4. Duplicate Finder

Finds duplicate files by size + name pattern in a directory tree. Review before deleting.

```

from pathlib import Path
from collections import defaultdict

ROOT = Path("./media")
by_key = defaultdict(list)

for f in ROOT.rglob("*"):
    if f.is_file():
        key = (f.name.lower(), f.stat().st_size)
        by_key[key].append(f)

for (name, size), paths in by_key.items():
    if len(paths) > 1:
        print(f"Possible duplicates of {name} ({size} bytes):")
        for p in paths:
            print(" ", p)

```

5. Media Metadata Helper

Prints basic file info (size, modified time, extension) for every media file in a folder. Extend later with mutagen / Pillow if you need deeper tags.

```

from pathlib import Path
from datetime import datetime

FOLDER = Path("./downloads")
MEDIA = {".mp4", ".mov", ".jpg", ".jpeg", ".png", ".webp", ".mp3"}

for f in sorted(FOLDER.iterdir()):
    if f.suffix.lower() in MEDIA and f.is_file():
        st = f.stat()
        mtime = datetime.fromtimestamp(st.st_mtime).strftime("%Y-%m-%d %H:%M")
        size_mb = st.st_size / (1024 * 1024)
        print(f"{f.name:40} {size_mb:6.2f} MB {mtime}")

```

Notes

- Always work on copies first.
- These scripts are intentionally minimal so you can read and adapt them.
- For production pipelines add logging, dry-run flags and error handling.

Related tool: Local Social Media Downloader PRO is a small local Python utility for supported Instagram / TikTok / Pinterest workflows (\$10 one-time). <https://digitaltoolsstudios.gumroad.com//localsocialdownloader> — Anton Glaser, DigitalToolsStudios